A Survey of AIS-20/31 Compliant TRNG Cores Suitable for FPGA Devices

Oto Petura, Ugo Mureddu, Nathalie Bochard, Viktor Fischer, Lilian Bossuet
Hubert Curien Laboratory, UMR 5516 CNRS,
Jean Monnet University Saint-Etienne
18, rue Pr. Lauras, 42000 Saint-Etienne, France
email: (oto.petura, ugo.mureddu, nathalie.bochard, fischer, lilian.bossuet)@univ-st-etienne.fr

***Abstract*—FPGAs are widely used to integrate cryptographic primitives, algorithms, and protocols in cryptographic systems-on-chip (CrySoC). As a building block of CrySoCs, True Random Number Generators (TRNGs) exploit analog noise sources in electronic devices to generate confidential keys, initialization vectors, challenges, nonces, and random masks in cryptographic protocols. TRNGs aimed at cryptographic applications must fulfill the security requirements defined in the German Federal Bureau for Information Security's (BSI) recommendations AIS-20/31, which has become a *de facto* standard in Europe. Many TRNG cores have already been published, only a few of which are suitable for FPGAs and even fewer comply with AIS-20/31. Here we present the results of the implementation of AIS-20/31 compliant TRNG cores in three FPGA families: Xilinx Spartan 6, Altera Cyclone V and Microsemi SmartFusion 2. In addition to common design parameters like area, bit rate and power/energy consumption, we compare and discuss the feasibility of generator cores in different FPGAs and the statistical quality of their output. These results will help designers select the best generator and the device family to match the requirements of the data security application. To ensure reproducibility of the results, the open source VHDL code of all generators adapted to individual families can be downloaded from the dedicated web page.**

I. Introduction

True Random Number Generators (TRNG) are used in cryptography to generate confidential keys, initialization vectors, challenges, nonces, and random masks in side channel attack countermeasures. They exploit intrinsic noise sources in electronic devices as a source of randomness.

FPGAs are widely used to integrate cryptographic primitives, algorithms and protocols in the cryptographic systems on chip (CrySoC). As a building block of the CrySoC, the TRNG must meet strict security requirements [1].

TRNGs are typically composed of an analog physical source of randomness, a digitizer, and an optional entropy conditioning block. The source of randomness, digitization, and the entropy harvesting mechanism depend to a large extent on the selected technology, a standard or even a recommended TRNG does not exist. Depending on the characteristics of the source of randomness and the quality of the digital noise (the output of the digitizer), designers select the entropy conditioning method that will enhance the statistical properties of generated numbers.

In the past, during the design and the security evaluation and certification process, the principle of the TRNG and its implementation were only evaluated statistically: the generated numbers were tested using standard test suites such as FIPS 140-1 [2], NIST SP 800-22 [3], DIEHARD [4], and DIEHARDER [5].

However, this approach is not suitable for modern data security systems for several reasons: 1) post-processing can mask considerable weaknesses in the source of randomness; 2) generic statistical tests can only evaluate the statistical quality of the numbers that are generated and not their entropy; 3) high-end standard statistical tests are complex and hence both expensive and slow, plus they require huge data sets. Consequently, they are only executed occasionally or on demand and only on selected sets of data of limited size.

The German Federal Office for Information Security recently proposed a methodology of evaluation of random number generators (AIS-20/31) [6], which should help designers to better consider security aspects in their design and help evaluators to rigorously evaluate the security of the generator during the certification process. Currently, all TRNGs aimed at cryptographic applications that require a security certificate for use in European union must comply with AIS-20/31.

Many TRNG cores have already been published, but only a few of them are suitable for FPGAs, and even fewer comply with AIS-20/31. Our aim was to select such generators and to fairly evaluate the difficulties related to their implementation in different FPGA technologies, their area, output bit rate, power requirements, and the statistical quality of their output.

To compare TRNG principles and their implementations in different FPGA families as fairly as possible, the evaluation boards should have the same topology and should use as few components introducing deterministic noise as possible (e.g. should be powered using low noise power supplies). This rigorous approach was not applied in recent papers [7], [8], [9], [10], [11], in which five different retail evaluation boards, which contain switching power supplies, were used. This means the claimed performance of TRNGs evaluated in the above mentioned papers was specific to particular boards and operating conditions.

Our contribution: 1) We implement AIS-20/31 compliant TRNG cores in different FPGA families; 2) we compare the generators using always the same low noise dedicated hardware; 3) we propose two new evaluation criteria for better comparison of TRNGs – the energy efficiency and the product of the entropy & bit rate; 4) to ensure reproducibility of

Authorized licensed use limited to: INSTITUTO NAL. ASTROFISICA O. Y ELECT (INAOE). Downloaded on October 05,2021 at 19:10:45 UTC from IEEE Xplore. Restrictions apply.

our results, we have made the VHDL code of all generators adapted to individual families freely available (open source).

The paper is organized as follows. In Section II, we describe how we selected AIS-20/31 compliant TRNG cores that are suitable for implementation in FPGAs. In Section III, we describe the strategy of implementation and evaluation of the TRNG cores. In Section IV, we describe the implementation of selected generators in the selected FPGA families. In Section V, we discuss the results and propose selection criteria to help designers select an appropriate design in the future. Section VI concludes the paper and describes the future outlook.

II. Selection of TRNG Cores

Many TRNG cores have been published up to now, but many are not suitable for implementation in digital devices like FPGAs. This is the case of generators using analog components like analog amplifiers [12] or analog to digital converters used in chaos-based TRNGs [13], [14]. Furthermore, some digital designs require full custom digital technology and are thus not feasible in FPGAs [15], [16].

On the other hand, some generators use the specific features of specific FPGA family and are not directly feasible in other FPGA families [17], [18]. Our objective was to select sufficiently generic principles that are feasible in all recent FPGA families.

As explained in the previous section, TRNGs that are intended to be used in cryptographic applications must comply with the AIS-20/31 standard. Such TRNG cores must fulfill several requirements: 1) their design must be simple and comprehensible, the source of randomness must be clearly defined; 2) the underlying random process must be stationary and the stochastic model must be feasible; 3) the raw binary signal must be available for further off-line and on-line testing. In addition, it is helpful if the source of randomness (e.g. the clock jitter coming from the thermal noise) is quantifiable – its measurement inside the device can provide a basis for fast and efficient embedded statistical tests.

Several other generators, which were or could be implemented in FPGAs, do not comply with the AIS-20/31 standard. This is the case of generators which intrinsically combine true randomness and pseudo-randomness in a complex system for which the stochastic model is not feasible or at least not plausible [18], [19], [20].

We pre-selected TRNG cores that use oscillating circuitries: single-event ring oscillators (i.e. standard ring oscillators) [21], [9], [22], multi-event ring oscillators with signal collisions (i.e. transition effect ring oscillators) [7], multi-event ring oscillators without signal collisions (i.e. self-timed rings) [23], and phase-locked loops (PLLs) [24]. Consequently, all of them should be feasible in recent and future families of FPGAs. They all use simple and comprehensible sources of randomness, their raw random signal is available outside the generator, and the stochastic model of the generator exists or is feasible. Therefore, we can conclude that all of them comply with the AIS-20/31 requirements.

III. Strategy for the Implementation and Evaluation of TRNG Cores

Our objective was to use the same hardware configuration for all TRNG cores and for different FPGA families. The hardware/software system we used had three components: an FPGA device with the target of evaluation (TOE), the acquisition board and the PC running the software. The TOE implemented in FPGA devices was connected to the acquisition card using a simple serial interface – a serial data stream and a data strobe signal was sent to the acquisition board via two low voltage differential signaling (LVDS) links. The generated bit streams were saved in a 4-MB SRAM memory of the acquisition card and sent to the PC using the USB bus.

This strategy has several advantages: 1) the data interface of the TOE is very simple and its impact on the operation of the TRNG core is reduced to the minimum; 2) because of the use of the LVDS links, the TOE can be placed relatively far from the acquisition card (e.g. in a Faraday cage) and the data transfer is faster and more robust; 3) the use of the 4-MB memory guarantees undisrupted data transfers from the TOE to the PC (note that the USB bus cannot guarantee continuity of data transfers).

To reduce the vulnerability of the generators to external manipulations, we did not use external clocks: all the clock signals were generated inside the TOE, for example, using a ring oscillator with appropriate topology.

We preselected three representative FPGA families: Xilinx Spartan 6, a 45 nm SRAM-based FPGA family using 6-input look-up tables (LUTs); Altera Cyclone V, a 28 nm SRAM-based FPGA using 6-input LUTs; and Microsemi SmartFusion 2, a 65 nm FLASH based FPGA using 4-input LUTs.

Since expressing logic area in slices or adaptive logic modules (ALMs), as made often by FPGA vendors, would not allow the fair comparison of designs, we characterize the area of generators using the number of occupied look-up tables (LUTs) and registers.

One of the parameters used for design evaluation is power consumption. The power consumption of TRNGs is relatively low and is mostly comparable to, or even lower than, the standby power consumption of an empty device. For this reason, we first implemented a reference design in which an input static signal just crossed the device and only an output multiplexer was implemented inside it (the same multiplexer was used later to keep the generator running, while blocking its output to the input/output circuitry). With this small reference project the Spartan 6, Cyclone V, and SmartFusion 2 devices consumed 3.5 mW, 29.7 mW, and 12.5 mW, respectively. This power was subtracted from the total power consumption measured in all the experiments. The results presented in the following sections are thus the net power consumption of the designs we tested.

In most TRNGs, we use the jitter of the clock signal generated in ring oscillators as a source of randomness. Therefore, we first characterized the period jitter of the generated clock signal depending on its frequency. Only one ring was

Authorized licensed use limited to: INSTITUTO NAL. ASTROFISICA O. Y ELECT (INAOE). Downloaded on October 05,2021 at 19:10:45 UTC from IEEE Xplore. Restrictions apply.

implemented in FPGA and the period jitter was measured at an LVDS output of the device using a Lecroy WaveRunner 640ZI oscilloscope (4GHz bandwidth, 40 GS/s) with a D420 WaveLink 4 GHz differential probe. The results of the jitter measurement are presented in Fig. 1. It can be seen that the Spartan 6 and Cyclone V families feature comparable period jitter ranging from 2 to 4 ps for clock periods between 4 and 8 ns. The SmartFusion 2 family features the period jitter between 8 and 10 ps for the same range of clock periods. In this family, for longer clock periods, the period jitter increases considerably, probably because of some global deterministic jitter (e.g. the jitter coming from the embedded RC oscillator). In all measurements, for clock periods under 3 ns, the noise of the measurement equipment starts to dominate.

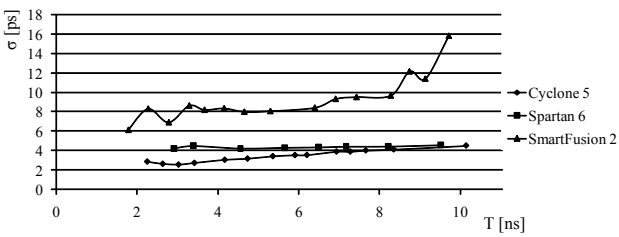

Fig. 1. Results of the period jitter measurements in the selected FPGA families

To evaluate the statistical quality of the generated numbers, we used Procedure B of the AIS-20/31, which is designed to test raw random signals. We used test T8 of this procedure for a rough estimation of the entropy rate. For a rigorous entropy estimation a stochastic model of the generator should be used. This is out of the scope of our paper.

Since all the presented TRNG designs have many degrees of freedom, in our comparisons, we chose the parameters (number of delay elements, division factors, etc.) giving the highest entropy rate at the output.

IV. Implementation of Selected TRNG Cores in FPGA

A. Ring Oscillator Based Elementary TRNG

The ring oscillator based elementary TRNG (ERO-TRNG) was prosed and modeled in [21]. The block diagram of the ERO-TRNG as implemented in FPGAs is depicted in Fig. 2.

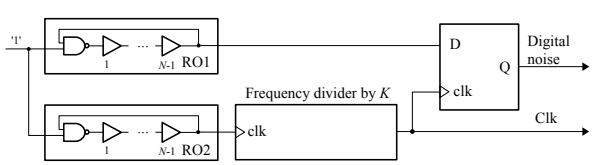

Fig. 2. Architecture of the ERO-TRNG core

The generator uses two identical ring oscillators (RO1 and RO2) as sources of randomness. The output of one ring (a jittery clock signal) is sampled in a D flip-flop after a sufficiently long accumulation period derived from the second clock signal using a frequency divider by K (a 17-bit synchronous counter). Let us note that thanks to the use of two identical oscillators, the impact of the global sources of randomness, which can be easily manipulated, is significantly reduced.

The number of elements of the ring (N) was chosen to get approximately the same clock frequency (300 MHz) in all families: 3 elements (one NAND gate and 2 buffers) were used in Spartan 6 and 5 elements in Cyclone V and SmartFusion 2 family. The sampling period (and hence parameter K) was selected depending on the clock frequency and the size of the jitter (see Fig. 1).

The lower entropy bound defined in [21] can be adapted to the elementary TRNG from Fig. 2 as:

$$H_{min} = 1 - \frac{4}{\pi^2 ln(2)} e^{\frac{-\pi^2 \sigma_{th}^2 K T_2}{T_1^3}}, \qquad (1)$$

where σ_{th}^2 is the variance of the jitter due to thermal noise, K is the frequency division factor and T_1, T_2 are periods of the clock signals generated by RO1 and RO2, respectively. Let us note that the oscillation frequency and the size of the jitter differ in each FPGA family, consequently parameter K and hence the output bit rate also differ.

Clock periods of the ring oscillators were about 3 ns in all the devices. The total period jitter exploited in the TRNGs was then approximately 4 ps, 3 ps, and 8 ps for the Spartan 6, Cyclone V, and SmartFusion 2 device, respectively. The frequency division factor K was set up according to Eq. (1) to 80 000, 135 000, and 20 000, respectively.

The two ring oscillators were placed and routed manually in order to ensure the repeatability of the design. Although both rings were placed in close vicinity, apparently, they did not lock.

1) Results and Discussion on the ERO-TRNG: Results of the implementation of the ERO-TRNG are presented in Table II. The area of the TRNG is relatively small at the expense of a low output bit rate.

The rings should have the same topology in order to compensate for the impact of the global jitter sources on the generator. This requires manual placement (and eventually routing) of the rings, which is a relatively simple operation.

The ERO-TRNG has very small output bit rate, but also a very high security potential – to guarantee a sufficient entropy rate for the given division factor K, it is sufficient if the embedded test checks that the rings are oscillating and that they are not locked. Note that solid stochastic model exists for this TRNG and the generator thus complies with AIS-20/31.

A higher clock frequency increases the output bit rate at the cost of higher power consumption. Since at any time, only one event (rising or falling edge) propagates in the ring, the power consumption of the ring does not depend on the number of delay elements and is thus the same for all rings. Therefore, the higher power consumption of the TRNG is not caused by the ring itself, but rather by the frequency of the clock signal used in the counter.

B. Ring Oscillator Coherent Sampling Based TRNG

The coherent sampling ring oscillator based TRNG (COSO-TRNG) was first proposed in [9]. The block diagram of the

Authorized licensed use limited to: INSTITUTO NAL. ASTROFISICA O. Y ELECT (INAOE). Downloaded on October 05,2021 at 19:10:45 UTC from IEEE Xplore. Restrictions apply.

COSO-TRNG core implemented in our devices is depicted in Fig. 3.

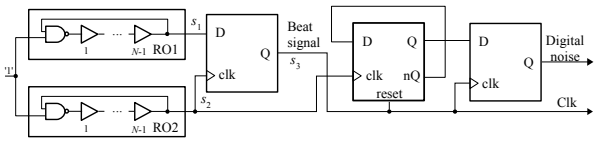

Fig. 3. Architecture of the COSO-TRNG core

The generator uses two oscillators, which have the same number of elements and their topology (placement of the delay elements) is also the same. The clock signal s_1 was sampled in a D flip-flop on rising edges of the clock signal s_2. The resulting beat signal was then used to flip the T flip-flop (a 1-bit counter). To extract randomness from the jitter, the difference between two clock periods must fulfill the following condition:

$$\Delta_T < \sqrt[3]{\sigma_T^2 \cdot T} = \Delta_{T_{max}}. \quad (2)$$

Fulfilling this condition is not an easy task. We measured the clock period T and the standard deviation of the period jitter to compute $\Delta_{T_{max}}$. Then we tried different placements and routings to find Δ_T smaller than $\Delta_{T_{max}}$. In our case, we obtained satisfying results with:

- N = 8 in Spartan 6 giving T = 6.92 ns, $\sigma \sim$ 4 ps, $\Delta_{T_{max}} \sim$ 50 ps
- N = 6 in Cyclone V giving T = 3.17 ns, $\sigma \sim$ 2.5 ps, $\Delta_{T_{max}} \sim$ 30 ps
- N = 10 in SmartFusion 2 giving T=5.4 ns, $\sigma \sim$ 8 ps, $\Delta_{T_{max}} \sim$ 70 ps

1) Results and Discussion on the COSO-TRNG: Results of the implementation of the COSO-TRNG are presented in Table II. As expected, the area of the TRNG is very small, while the output bit rate can be relatively high. When setting the size of Δ_T, a trade off between the entropy rate and the output bit rate must be made: a small Δ_T increases the entropy rate, but decreases the bit rate and vice versa.

The main difficulty in designing the COSO-TRNG is the need to set parameter Δ_T precisely – the difference in the output periods must be sufficiently small, i.e. comparable in size to the accumulated jitter. Unfortunately, even rings that have exactly the same topology can generate clock signals with periods that differ too much because of the dispersion of electrical parameters inside the device.

To obtain two periods that are sufficiently close, we placed one oscillator in a stable position inside the FPGA and the other ring was moved automatically (using a script written in the TCL language) to different places inside the device until the difference in period between the two generated clock signals was sufficiently small. Once a convenient difference in periods was obtained, the placement and routing constraints of the two oscillators were saved to prevent any changes caused by future recompiling of the project.

Even if the process of finding the right solution is further optimized, the results will remain device dependent – the optimization process must be repeated for each individual device. This makes the practical use of the generator questionable, if some other automatic way of setting the clock frequencies is not found.

Concerning security, since both rings have the same topology (this is a must to reduce the difference in the period), the impact of the global jitter sources on the generator is significantly reduced. Higher clock frequency increases the output bit rate, while the power consumption does not change significantly.

C. Multi-Ring Oscillator Based TRNG

The multi-ring oscillator based TRNG (MURO-TRNG) and its stochastic model were originally proposed in [22]. The block diagram of the MURO-TRNG core architecture implemented in our devices is depicted in Fig. 4.

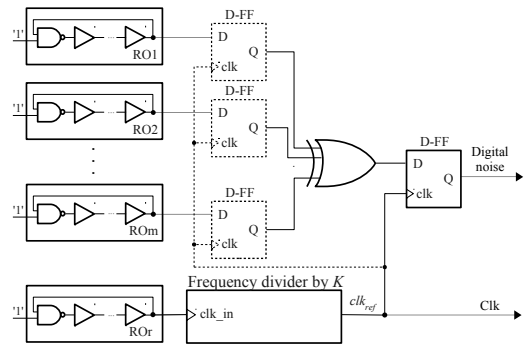

Fig. 4. Architecture of the MURO-TRNG

The generator uses m ring oscillators as sources of randomness. Assuming that the oscillators are independent, their phase is uniformly distributed. Based on the assumption of uniformity, the number of rings must fulfill the following condition:

$$m > \frac{T}{\sigma_{acc}}, \quad (3)$$

where T is the mean value of the clock period and σ_{acc} is the standard deviation of the jitter accumulated during the sampling period. Since the clock phases are uniformly distributed, the probability that the D flip-flop at the output of the generator will sample some clock edges (out of m edges theoretically available at the XOR gate output) is also uniformly distributed.

However, the authors of [8] showed that output of a single m-input XOR gate cannot follow too many high-speed input signals. They proposed using additional flip-flops (dashed lines in Fig. 4), which resolved the problems concerning the speed of the XOR gate. Since the authors of [25] proved that the stochastic model remains valid, we used this modified MURO-TRNG architecture in our study.

1) Results and Discussion on the MURO-TRNG: Results of the implementation of the MURO-TRNG are listed in Table II. The rings were built using four delay elements (one NAND gate and three buffers) and they oscillated at frequencies between 200 MHz and 350 MHz depending on the device and the placement of rings. We sized this TRNG core for

Authorized licensed use limited to: INSTITUTO NAL. ASTROFISICA O. Y ELECT (INAOE). Downloaded on October 05,2021 at 19:10:45 UTC from IEEE Xplore. Restrictions apply.

the Cyclone V device, which generates the smallest jitter. In the given frequency range, the standard deviation of the period jitter was approximately $\sigma_{per} = 3$ ps (see Fig. 1) and according to Eq. (3), the generator should have more than 1200 rings.

To reduce the number of rings, we accumulated the jitter during $K = 100$ periods of the reference clock signal. The accumulated jitter was therefore $\sigma_{acc} = 30$ ps and the number of rings according to Eq. (3) should be equal or greater than 120 which is the number used in our implementation. This value is comparable with that published in [22] ($N = 114$). Nevertheless, the generator occupies a very big area.

Another problem with this generator is that some rings can (and probably will) lock to each other and deteriorate the distribution of events, which will no longer be uniform. This can in turn reduce the entropy rate at the generator output. Unfortunately, it is practically impossible to check if some rings among such a huge number of rings are locked. Last but not least, because of this huge number of rings, power consumption is considerable.

On the contrary, the MURO-TRNG has the following advantages: it does not need manual placement and routing, the output bit rate and the entropy rate are very high.

D. Coherent Sampling Based TRNG Using PLLs

The coherent sampling based TRNG which uses PLLs (PLL-TRNG), was first published in [24] and the model of the generator was proposed in [26]. The block diagram of the PLL-TRNG core implemented in our devices is depicted in Fig. 5.

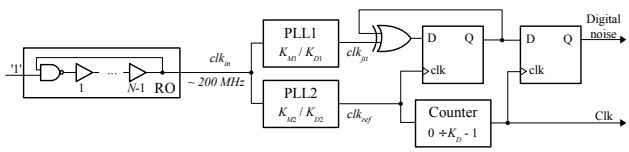

Fig. 5. Architecture of the PLL-TRNG core

The generator is based on the fact that using PLLs, the frequencies of two generated clock are mutually related. Since $f_{jit} = f_{in} \cdot K_{M1}/K_{D1}$ and $f_{ref} = f_{in} \cdot K_{M2}/K_{D2}$, the relationship between f_{jit} and f_{ref} is as follows:

$$f_{jit} = f_{ref} \cdot \frac{K_{M1}}{K_{D1}} \cdot \frac{K_{D2}}{K_{M2}} = f_{ref} \cdot \frac{K_M}{K_D}. \tag{4}$$

Thanks to the coherent sampling principle, the samples of the jittery clock signal obtained in the D flip-flop at the rising edges of the reference clock signal are uniformly distributed over the translated period T_{jit}. The distance between the samples is then $\Delta = T_{jit}/K_D$ and K_D samples must be XOR-ed to obtain one output bit (see Fig. 5).

The output bitrate R of the generator and the sensitivity to the jitter S are defined as:

$$R = f_{ref}/K_D, \tag{5}$$

$$S = \Delta^{-1} = K_D/T_{jit}. \tag{6}$$

To obtain high entropy random bits, the distance between the samples Δ must fulfill the following condition:

$$\Delta \ll \sigma_r, \tag{7}$$

where σ_r is the relative jitter between the two generated clocks. According to Eq. (5) and (6), the objective of a designer is to make Δ as small as possible, while maintaining the output bit rate (R) in an acceptable range by setting the input frequency (f_{in}) and the multiplication and division factors of both PLLs.

We followed the same strategy to determine the multiplication and division factors for given families. The frequency of the input clock signal generated by a ring oscillator was approximately 200 MHz in all cases. The parameters of PLLs and the distance between samples (Δ) are presented in Table I.

TABLE I
PARAMETERS OF PLLS AND CORRESPONDING DISTANCE BETWEEN SAMPLES (Δ) IN SELECTED FPGA FAMILIES

FPGA	PLL1		PLL2		Total		Δ
	K_M	K_D	K_M	K_D	K_M	K_D	[ps]
Spartan 6	37	17	17	7	1 377	259	4.82
Cyclone V	31	29	23	18	667	558	4.25
SmartFusion2	74	162	18	22	729	407	9.10

1) Results and Discussion on the PLL-TRNG: Results of the implementation of the PLL-TRNG are presented in Table II. Besides setting the PLL parameters as presented in the previous section, the main difficulty in designing the PLL-TRNG in FPGAs is related to the routing of clock signals from the logic area to the PLL and vice versa, since we wish to generate the PLL input clock signal using a ring oscillator inside the logic area and the two clock signals generated in two PLLs must be available in the same clock domain (the domain in which the D flip-flop is situated).

We were able to internally route the clock signal generated by the ring oscillator directly to the PLL input in all three FPGA families. However, in the Spartan 6 family, the outputs of the two PLL blocks are routed via dedicated clock wires into different clock regions. The easiest solution was to use a non-dedicated clock path constraint that forced the router to route the PLL output signal via general purpose interconnection wires instead of the dedicated clock paths.

We ensured that only the clk_{jit} (i.e. the sampled clock) signal was routed this way and the clk_{ref} (the sampling clock) was routed via a standard dedicated clock path. Designers need to be aware that this kind of routing could be more sensitive to ambient noises and it could thus add more unwanted (manipulable) jitter to the clock signal.

Fortunately, PLL blocks in Altera and Microsemi FPGAs do not have this kind of constraint and the PLL-TRNG was easier to implement in these FPGAs.

PLLs are expensive hardware blocks, since they occupy huge silicon area. However, in the context of FPGAs, they are available 'for free'. On the other hand, once used in the

Authorized licensed use limited to: INSTITUTO NAL. ASTROFISICA O. Y ELECT (INAOE). Downloaded on October 05,2021 at 19:10:45 UTC from IEEE Xplore. Restrictions apply.

TRNG, they will not be available for the rest of the design. This disadvantage can be reduced by sharing at least one PLL between the TRNG and the application design. In our experience, this is always possible.

Aside from the area occupied by PLLs, the additional area used by the TRNG is very small and is mainly occupied by the ring oscillator which generates the internal clock signal.

Since voltage controlled oscillators (VCO) oscillate at high frequencies, the power consumption of the TRNG is relatively high. Let us note that in some families (e.g. those of Altera), the PLLs cannot be stopped and their VCOs oscillate even if the PLLs are not being used (instantiated).

The generator has two main advantages: it does not need manual placement and routing and the source of randomness is very well isolated from the rest of the device, since it has a separate power supply.

E. Transition Effect Ring Oscilaltor Based TRNG

The transition effect ring oscillator based TRNG (TERO-TRNG) was proposed in [7] and its stochastic model in [27]. The block diagram of the TERO-TRNG core implemented in our FPGA devices is depicted in Fig. 6.

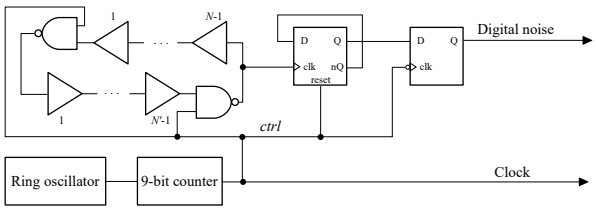

Fig. 6. Architecture of the TERO-TRNG core

The transition effect ring oscillator (TERO) is a multi-event ring oscillator with collisions built as a loop of logic gates. The loop contains an even number of inverting gates and any number of non-inverting gates. Because of the even number of inverting gates, the oscillator has to be restarted regularly – the two events created after each restart circulate inside the loop until a collision occurs, during which the edge that moves faster reaches the slower one.

The difference in the speed of circulating events is caused by differences in delays between inverters in loop branches and by analog phenomena in inverters and buffers. The circulating events create temporary oscillations that disappear after the collision.

The heart of the TERO-TRNG is the TERO cell, which is followed by a counter (in Fig. 6, the counter is represented by a T flip-flop) and an output data register.

The output of the counter represents realizations of the random variable (i.e. the number of oscillations in subsequent control periods). The control signal, which periodically restarts the TERO cell, is generated using a conventional ring oscillator. The control signal defines the output bit rate of the generator.

1) Results and Discussion on the TERO-TRNG: Results of the implementation of the TERO-TRNG are presented in Table II. We built the TERO cell using 10 buffers and one NAND gate in both TERO branches. The control signal was generated with a ring oscillator configured to oscillate at 150 MHz connected to a 9-bit synchronous counter. The frequency of oscillations of the TERO cell was approximately 90 MHz for Spartan 6 and 150 MHz for Cyclone V and SmartFusion 2 family.

The area of the TRNG core is very small, while the output bit rate can be relatively high. The architecture of the TRNG is simple, but to achieve a sufficient entropy rate, the two TERO cell branches must be unbalanced in such a way that the number of oscillation periods M fulfills the following condition:

$$100 < M < \frac{T_{meas}}{T_{osc}}, \tag{8}$$

where T_{meas} is the time of the measurement (a portion of the control period) and T_{osc} is the period of oscillations. This is difficult to obtain repeatedly – even devices configured with the same configuration file can give very different results. A perfectly balanced TERO cell would oscillate permanently and a very unbalanced one will feature very few oscillation periods.

The number of delay elements does not have a direct impact on power consumption. Because in each TERO configuration two events (two rising edges or falling edges) pass across the TERO cell at any time. However, it appears that a higher number of delay elements makes the TERO cell design easier – the balance can be adjusted in smaller steps.

F. Self Timed Ring Based TRNG

The self timed ring (STR) is a multi-event oscillator without signal collisions. The first TRNG using STRs was proposed in [23] and its model in [28]. The block diagram of the STR-TRNG core implemented in our devices is depicted in Fig. 7.

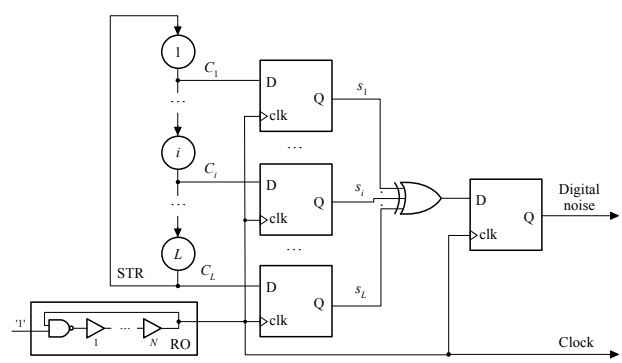

Fig. 7. Architecture of the STR-TRNG core

The STR is composed of L stages, each consisting of a Muller gate and an inverter. The STR stages communicate using the two-phase handshake protocol. In contrast to inverter ring oscillators, several events can propagate without colliding

Authorized licensed use limited to: INSTITUTO NAL. ASTROFISICA O. Y ELECT (INAOE). Downloaded on October 05,2021 at 19:10:45 UTC from IEEE Xplore. Restrictions apply.

thanks to this handshake protocol, which enables precise, built-in phase control of the internal clock signals.

The ring is initialized with E events which start propagating during a transient state. Independently of their initial positions and thanks to two analog mechanisms inferred in the ring (the Charlie and the drafting effects), they end up in a steady state.They either: form a cluster which propagates in the ring (burst oscillation mode), or spread out around the ring and propagate with a constant temporal spacing (evenly-spaced oscillation mode). Both these oscillation modes are stable and depend on the static parameters of the ring (mainly the ring occupancy $E/(L-E)$ with respect to the ratio of forward and reverse static delays).

If E events are confined in L stages and spread evenly around the ring, the phase shift between two stages separated by n stages is [23]:

$$\varphi_n = n \times \frac{E}{L} \times 180. \tag{9}$$

If the number of events and the number of stages are co-prime, the STR exhibits as many different equidistant phases as the number of stages. In this case, if T is the oscillation period, the phase resolution can be expressed as [28]:

$$\triangle\varphi = \frac{E}{2L}. \tag{10}$$

The oscillation period of an STR does not depend on the number of its stages, but on the number of propagating events. It is therefore possible to increase the number of ring stages (L) while keeping the frequency constant. Consequently, the phase resolution of an STR can theoretically be set as finely as needed.

In our STR-TRNG, the sampling clock was generated by an additional ring oscillator. Each STR stage was sampled in the D flip-flop. The outputs of the flip-flops were XOR-ed together and the output of the XOR gate was sampled again in another D flip-flop at the same frequency. To guarantee a sufficient entropy rate at the output of the generator, the following condition must be fulfilled:

$$\triangle\varphi < \sigma_{acc}, \tag{11}$$

where $\triangle\varphi$ is the phase resolution defined in Eq. (10) and σ_{acc} is the standard deviation of the accumulated jitter.

1) Results and Discussion on the STR-TRNG: Results of the implementation of the STR-TRNG are presented in Table II. Although the implementation of the Muller gate in LUT based FPGAs is relatively simple, the STR-TRNG needs careful placement and routing to guarantee the evenly spaced mode (and hence precise timing of events) and also to maintain the frequency as high as possible. Since the number of events circulating inside the ring is constant, the entropy rate can be increased by increasing the frequency of the clock signal, determined by the longest delay between Muller gates of the ring.

Our implementations give the STR frequency of around 300 MHz. The smallest jitter (in Cyclone V) at this frequency is around 3 ps. According to Eq. (10) and (11) we should have $L > 550$. The corresponding STR would occupy a very big area. Therefore, we decreased the length of the STR to 255.

Table II shows that the output bit rate of the STR-TRNG is extremely high at the expense of high power consumption. The entropy & bit rate product is also very high. According to Eq. (11), the entropy could be further increased by increasing the number of STR stages and the number of circulating events, while maintaining the evenly spaced mode. However, our practical experience showed that if the STR has too many stages the evenly spaced mode is difficult to obtain. In our case an STR with 255 stages used seemed to be a reasonable compromise that works in all selected FPGA families.

V. Discussion on FPGA Implementation of Selected TRNG Cores

In the following paragraphs, we evaluate and briefly discuss the first results obtained in three selected FPGA families. We use the following TRNG characteristics:

- *Area* – the total area is expressed as the number of LUTs and registers occupied by the design.
- *Power consumption* in mW – this parameter gives the power consumption of the core of the TRNG (without data interface).
- *Bit rate* in Mbits/s – since the data interface of the acquisition card is faster than all available TRNGs, the speed of the acquisition card does not limit the speed of data transfer and the output bit rate obtained only depends on the generator.
- *Energy efficiency* in bits/μWs – this parameter specifies the relationship between the bit rate (in bits/s) and power consumption (in μW). In other words, it gives the number of bits obtained per energy unit (one μWs).
- *Entropy rate* per output bit – entropy is estimated using the AIS-31 Procedure B, test T8.
- *Product of entropy & bit rate* – since the entropy rate and the bit rate at the output of the generator are closely linked (the output with a low entropy rate and high bit rate can be post-processed to increase entropy at the expense of a smaller bit rate), they should be evaluated together using the same parameter – the product of the entropy rate and of the bit rate.
- *Feasibility and repeatability* – this evaluation parameter reflects the difficulty of the design and its repeatability across selected FPGA families. The feasibility and repeatability criterion is divided into six levels, which will be explained in the next few paragraphs.

The highest score (5) for the feasibility and repeatability will be given to designs that do not need any manual intervention and the obtained results are always satisfactory, independently from the family. The results are repeatable in all devices of the same family.

A score of 4 will be given to designs that need some simple manual setup (e.g. manual placement) and the obtained results are repeatable in all devices of the same family.

A score of 3 will be given to designs that need a manual setup (optimization of parameters) and some tricky setup in

Authorized licensed use limited to: INSTITUTO NAL. ASTROFISICA O. Y ELECT (INAOE). Downloaded on October 05,2021 at 19:10:45 UTC from IEEE Xplore. Restrictions apply.

some family (but the design is feasible in all families). This manual optimization cannot be automatically translated from one family to another, but remains the same for all devices of the same family (repeatability).

A score of 2 will be given to designs that necessitate the use of some complex manual setup depending on the family (optimization of the topology and routing). The obtained results are the same for all devices of the same family (repeatability).

A score of 1 will be given to designs that need manual setup for each device individually (even for the devices from the same family), but a satisfactory solution can always be obtained.

A score of 0 will be given to designs, in which satisfactory results cannot be guaranteed (they appear randomly).

Table II summarizes the results of the implementation of the selected TRNG designs. First of all, it can be observed that the area occupied by individual generators does not differ significantly between the families, regardless of the size of the LUT. This can be explained by the fact that each delay element of rings is implemented using exactly one LUT.

It can be observed that the ERO-TRNG core occupies very small area and consumes relatively little power. It is very easy to implement (highest feasibility and repeatability), but it has a very small bit rate.

The COSO-TRNG core occupies the smallest area and consumes the least power. At the same time, it has an interesting bit rate. It also reaches a high entropy rate and relatively high entropy & bit rate product. However, it is difficult to implement – it must be placed manually in each individual device (low feasibility and repeatability). This disadvantage becomes eliminatory in most practical applications.

The MURO-TRNG core is relatively easy to implement and it features relatively high bit rate at the cost of the area. It has the second highest entropy & bit rate product, but the energy efficiency is not remarkable.

The area occupied by the PLL-TRNG core seems to be small, however, the area occupied by the PLLs is not taken into account in Table II. The main advantage of this generator is related to the fact that PLLs are very well isolated from the rest of the device and therefore more robust.

The TERO-TRNG core seemed to be very promising, but the need of the manual set up of the TERO cell represents an important handicap and its weakest point.

We obtained very interesting results with the STR-TRNG independently from the family. This TRNG core has extremely high bit rate and a high entropy and thus also a very high entropy & bit rate product. While it has the highest power consumption, it maintains a very high energy efficiency. Unfortunately, it occupies huge area and it needs precise placement and routing.

When considering the power and energy consumption, the energy efficiency parameter can be very useful to estimate the energy that must be spent for generating one random bit. This is clearly visible in the case of the STR-TRNG.

On the other end, the use of the entropy & bit rate product does not seem to bring any significant advantage to our evaluation. However, this fact is caused by the strategy of our design: to obtain the highest entropy possible for each TRNG design. We are convinced that if the entropy rate per bit is not close to one (which is the case in many practical applications), the entropy & bit rate product can help in finding the compromise between the entropy rate and the bit rate.

If we compare individual generators from the point of view of different parameters, we can definitely observe that a generator giving the best results in all TRNG parameters does not exist. It can be seen that the ERO-TRNG core wins in feasibility and repeatability, but loses in the bit rate. On the other hand, the COSO-TRNG core obtains perfect results in area, but very bad score in feasibility and repeatability. The MURO-TRNG core can represent a compromise between the bit rate and feasibility, but can be weak from the security point of view – the rings can lock to each other and decrease significantly the entropy. The STR-TRNG core wins in the bit rate, energy efficiency, and entropy & bit rate product and it is certainly the best candidate for the high-speed applications, where the power consumption and difficulty of design are less important.

VI. Conclusions

In this paper, we presented and discussed implementation of selected TRNG cores in three different FPGA families. We showed that all cores comply with the stringent security requirements of the AIS-20/31 standard: they are simple and comprehensible, their stochastic model exists or at least is feasible, and the raw random signal is available for testing.

The results confirm that all the preselected TRNG designs are feasible in all selected families. However, two of evaluated design are not suitable for use in practice in their current form: the COSO-TRNG and the TERO-TRNG require some manual intervention (placement and routing) for each device individually.

The results also confirm that no ideal TRNG exists – the most suitable generator must be selected according to requirements of the data security application and some compromise must always be done.

It is important to stress that once the designer selects the appropriate generator core, he still has many degrees of freedom in the design and he can adapt the final choice of parameters to the practical needs of the application. The proposed TRNG evaluation parameters (energy efficiency and entropy & bit rate product) can be helpful in this task. Other combined metrics such as area & power consumption product can also be used. However it is more valuable for ASIC applications which we did not target.

Output parameters of all the tested generators, such as bit rate, power consumption, entropy, etc., depend on the underlying hardware to a great extent. Using the same evaluation boards in the same conditions is very important for a fair comparison.

Presented results, together with the VHDL codes which are freely available, can help designers to make their choice and

Authorized licensed use limited to: INSTITUTO NAL. ASTROFISICA O. Y ELECT (INAOE). Downloaded on October 05,2021 at 19:10:45 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SUMMARY OF IMPLEMENTATION RESULTS OF THE SELECTED TRNGS

TRNG type	FPGA device	Area (LUT/Reg)	Power cons. [mW]	Bit rate [Mbits/s]	Efficiency [bits/μWs]	Entropy per bit	Entropy * Bit rate	Feasib. & Repeat.
ERO	Spartan 6	46/19	2.16	0.0042	1.94	0.999	0.004	**5**
	Cyclone V	34/20	3.24	0.0027	0.83	0.990	0.003	
	SmartFusion 2	45/19	4	0.014	3.5	0.980	0.013	
COSO	Spartan 6	**18/3**	**1.22**	0.54	442.6	0.999	0.539	1
	Cyclone V	**13/3**	**0.9**	1.44	**1 600**	0.999	1.438	
	SmartFusion 2	**23/3**	**1.94**	0.328	169	0.999	0.327	
MURO	Spartan 6	521/131	54.72	2.57	46.9	0.999	2.567	4
	Cyclone V	525/130	34.93	2.2	62.9	0.999	2.197	
	SmartFusion 2	545/130	66.41	3.62	54.5	0.999	3.616	
PLL	Spartan 6	34/14	10.6	0.44	41.5	0.981	0.431	3
	Cyclone V	24/14	23	0.6	43.4	0.986	0.592	
	SmartFusion 2	30/15	19.7	0.37	18.7	0.921	0.340	
TERO	Spartan 6	39/12	3.312	0.625	188.7	0.999	0.624	1
	Cyclone V	46/12	9.36	1	106.8	0.987	0.985	
	SmartFusion 2	46/12	1.23	1	813	0.999	0.999	
STR	Spartan 6	346/256	65.9	**154**	**2 343.2**	0.998	**154.121**	2
	Cyclone V	352/256	49.4	**245**	**4 959.1**	0.999	**244.755**	
	SmartFusion 2	350/256	82.52	**188**	**2 286.7**	0.999	**188.522**	

test the designs on their own hardware. The VHDL source code of all the presented generators is freely available at [1].

We believe that most of our conclusions can be extended to implementation of presented TRNG cores to application specific integrated circuits (ASICs).

VII. ACKNOWLEDGEMENT

This work has received funding from the European Union's Horizon 2020 research and innovation programme in the framework of the project HECTOR (Hardware Enabled Crypto and Randomness) under grant agreement No 644052.

REFERENCES

[1] V. Fischer, "A closer look at security in TRNGs design," in *Proceedings of Constructive Side-Channel Analysis and Secure Design – COSADE'12*, ser. LNCS, vol. 7275. Springer-Verlag Berlin Heidelberg, 2012, pp. 167–182.

[2] NIST, "FIPS 140-1: Security Requirements for Cryptographic Modules," [online] Available from http://csrc.nist.gov/publications/fips/fips140-1/fips1401.pdf, 1994.

[3] A. Rukhin, J. Soto, J. Nechvatal, M. Smid, E. Barker, S. Leigh, M. Levenson, M. Vangel, D. Banks, A. Heckert, J. Dray, and S. Vo, "A statistical test suite for random and pseudorandom number generators for cryptographic applications, NIST Special Publication 800-22, revision 1a," [online] Available from http://csrc.nist.gov/publications/nistpubs/800-22-rev1a/SP800-22rev1a.pdf, 2010.

[4] G. Marsaglia, "DIEHARD: Battery of Tests of Randomness," [online] Available from http://stat.fsu.edu/pub/diehard/, 1996. [Online]. Available: http://stat.fsu.edu/pub/diehard/

[5] R. Brown, "Dieharder: A Random Number Test Suite," [online] Available from http://www.phy.duke.edu/rgb/General/dieharder.php, 2015.

[6] W. Killmann and W. Schindler, "A proposal for: Functionality classes for random number generators, version 2.0," 2011. [Online]. Available: https://www.bsi.bund.de/EN/Home/home_node.htm

[7] M. Varchola and M. Drutarovsky, "New high entropy element for FPGA based true random number generators," in *Cryptographic Hardware and Embedded Systems, CHES 2010*. Springer, 2010, pp. 351–365.

[1]https://labh-curien.univ-st-etienne.fr/cryptarchi/HECTOR_TRNG_designs

[8] K. Wold and C. Tan, "Analysis and enhancement of random number generator in FPGA based on oscillator rings," in *Proceedings of the International Conference on Reconfigurable Computing and FPGAs (ReConFig'08)*, 2008, pp. 385–390.

[9] P. Kohlbrenner and K. Gaj, "An embedded true random number generator for FPGAs," in *Proceedings of the 2004 ACM/SIGDA 12th international symposium on Field programmable gate arrays*. ACM, 2004, pp. 71–78.

[10] S. Yoo, B. Sunar, D. Karakoyunlu, and B. Birand, "A robust and practical random number generator," [online] Available from http://ece.wpi.edu/sunar/preprints/rings.pdf, 2007.

[11] M. Thamrin, I. Ahmad, and M. Hani, "A true random number generator for crypto embedded systems," in *Regional Postgraduate Conference on Engineering and Science*. School of Postgraduate Studies, UTM, 2006, pp. 253–256.

[12] B. Jun and P. Kocher, "The Intel random number generator," [online] Available from https://www.rambus.com/intel-random-number-generator/, 1999.

[13] S. Callegari, R. Rovatti, and G. Setti, "Embeddable ADC-based true random number generator for cryptographic applications exploiting nonlinear signal processing and chaos," *IEEE Transactions on Signal Processing*, vol. 53, no. 2, pp. 793–805, 2005.

[14] M. Drutarovsky and P. Galajda, "A robust chaos-based true random number generator embedded in reconfigurable switched-capacitor hardware," *Radioengineering*, vol. 16, no. 3, pp. 120–127, 2007.

[15] M. Bucci, L. Giancane, R. Luzzi, M. Varanonuovo, A. Trifiletti, I. AG, and A. Graz, "A novel concept for stateless random bit generators in cryptographic applications," *Proceedings of the IEEE International Symposium on Circuits and Systems (ISCAS 2006).*, p. 4, 2006.

[16] G. Taylor and G. Cox, "Behind Intels new random-number generator," [online] Available from http://spectrum.ieee.org/computing/hardware/behind-intels-new-randomnumber-generator, 2011.

[17] V. Rozic, B. Yang, W. Dehaene, and I. Verbauwhede, "Highly efficient entropy extraction for true random number generators on FPGAs," in *Proceedings of the 52nd Annual Design Automation Conference (DAC), San Francisco, USA*, 2015, pp. 116:1–116:6.

[18] T. Gyorfi, O. Cret, and A. Suciu, "High Performance True Random Number Generator Based on FPGA Block RAMs," in *Proc. Int. Symposium on Parallel and Distributed Processing*. IEEE, 2009, pp. 1–8.

[19] T. Tkacik, "A Hardware Random Number Generator," in *Cryptographic Hardware and Embedded Systems - CHES 2002*, ser. LNCS, vol. 2523, Redwood Shores, CA, USA. Springer Verlag, 2003, pp. 450–453.

Authorized licensed use limited to: INSTITUTO NAL. ASTROFISICA O. Y ELECT (INAOE). Downloaded on October 05,2021 at 19:10:45 UTC from IEEE Xplore. Restrictions apply.

[20] J. Golic, "New Methods for Digital Generation and Postprocessing of Random Data," *IEEE Transactions on Computers*, pp. 1217–1229, 2006.

[21] M. Baudet, D. Lubicz, J. Micolod, and A. Tassiaux, "On the security of oscillator-based random number generators," *Journal of Cryptology*, vol. 24, no. 2, pp. 398–425, 2011.

[22] B. Sunar, W. Martin, and D. Stinson, "A Provably Secure True Random Number Generator with Built-In Tolerance to Active Attacks," *IEEE Transactions on Computers*, pp. 109–119, 2007.

[23] A. Cherkaoui, V. Fischer, A. Aubert, and L. Fesquet, "A self-timed ring based true random number generator," in *IEEE International Symposium on Asynchronous Circuits and Systems (ASYNC 2013)*, 2013, pp. 99–106.

[24] V. Fischer and M. Drutarovsky, "True random number generator embedded in reconfigurable hardware," in *Proceedings of the International Workshop on Cryptographic Hardware and Embedded Systems (CHES 2002)*, ser. LNCS, vol. 2523, Redwood Shores, CA, USA. Springer Verlag, 2002, pp. 415–430.

[25] N. Bochard, F. Bernard, V. Fischer, and B. Valtchanov, "True-randomness and pseudo-randomness in ring oscillator-based true random number generators," *International Journal of Reconfigurable Computing*, vol. 879281, no. 2010, pp. 1–13, 2010.

[26] F. Bernard, V. Fischer, and B. Valtchanov, "Mathematical model of physical RNGs based on coherent sampling," *Tatra Mountains Mathematical Publications*, vol. 45, no. 1, pp. 1–14, 2010. [Online]. Available: http://tatra.mat.savba.sk/Full/45/01be-f-v.pdf

[27] P. Haddad, V. Fischer, F. Berdnard, and J. Nicolai, "A Physical Approach for Stochastic Modeling of TERO-based TRNG," in *Proceedings of the International Workshop on Cryptographic Hardware and Embedded Systems (CHES 2015), Saint-Malo, France*, ser. LNCS, vol. 9293. Springer Verlag, 2015, pp. 357–372.

[28] A. Cherkaoui, V. Fischer, L. Fesquet, and A. Aubert, "A very high speed true random number generator with entropy assessment," in *Cryptographic Hardware and Embedded Systems (CHES 2013)*, ser. LNCS, G. Bertoni and J.-S. Coron, Eds., vol. 8086. Springer, 2013, pp. 179–196.

Authorized licensed use limited to: INSTITUTO NAL. ASTROFISICA O. Y ELECT (INAOE). Downloaded on October 05,2021 at 19:10:45 UTC from IEEE Xplore. Restrictions apply.